

Mission Mini IDS 결과 보고서

Scapy 기반 경량 침입 탐지 시스템 — Port Scan · ARP Spoofing · 특정 패킷(DNS/ICMP/HTTP/SSH) 탐지

과목 Network Security Study 3·4주차 최종 과제 작성자 제출일 2026. 09. 27.

소스 코드 github.com/<your-id>/mini-ids

1. 개요

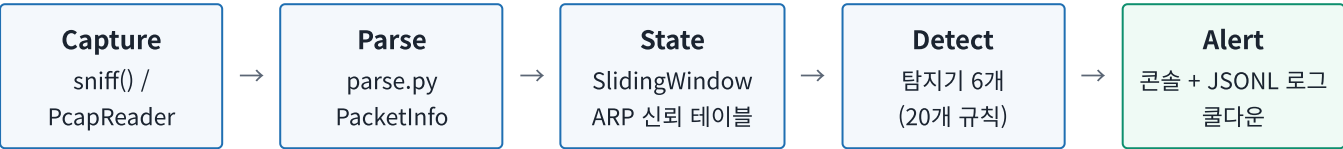
3·4주차 세미나의 핵심 질문인 "무엇을 보면 공격을 알아차릴 수 있는가?"를 코드로 옮기는 것이 목표였다. 1주차(Port Scan)와 2주차(Scapy로 패킷 필드 읽기)에서 배운 내용을 3·4주차의 **상태 저장 + 비교** 개념과 합쳐, 슬라이드 18의 **Capture → Parse → State → Detect → Alert** 구조를 그대로 가진 IDS를 만들었다.

필수 조건은 3개 중 2개 이상이었지만 **세 항목을 모두 구현**했다. 탐지 규칙은 총 20개다. 25개의 테스트 시나리오(공격 18, 정상 7)로 FP/FN을 측정했고, 일부 시나리오는 **일부러 탐지가 실패하도록** 만들어 한계를 확인했다.

주차 계획	실제 진행 내용
1주차 · 탐지 대상 선정	세 항목 모두 선정. Port Scan은 1주차의 SYN/3-way handshake, ARP는 3·4주차의 IP-MAC 매핑, 특정 패킷은 2주차의 DNS/HTTP/ICMP 필드 분석과 연결
2주차 · 패킷 필드 추출	<code>parse.py</code> : Scapy 패킷에서 필요한 필드만 뽑아 <code>PacketInfo</code> 로 정리 (아래 표)
3주차 · 탐지 로직 구현	슬라이딩 윈도우(시간 창), ARP 신뢰 테이블, 시그니처 패턴, 경고 쿨다운 구현
4주차 · 테스트와 한계 분석	Packet Crafting으로 정답이 붙은 PCAP 25개 생성 → TP/FP/FN/TN 자동 집계, 실시간 캡처 테스트, 단위 테스트 21개

개발 환경: Python 3.11, Scapy(GitHub 최신 소스, 2.6 계열) · 실행: `python3 ids.py -r file.pcap` 또는 `sudo python3 ids.py -i eth0`

2. 설계



탐지기는 Scapy 객체를 직접 뒤지지 않고 `PacketInfo` 만 본다. 이렇게 파싱과 탐지를 분리하면 새 규칙을 추가하기 쉽고, 단위 테스트도 간단해진다. 실시간 캡처든 PCAP 재생이든 같은 `MiniIDS.process()` 를 거친다. 시간은 벽시계가 아니라 **패킷의 캡처 시각(pkt.time)**을 기준으로 한다. 그래서 PCAP을 빠르게 재생해도 "10초 안에"라는 조건이 원래 시간대로 계산된다.

탐지에 필요한 패킷 필드 (2주차 단계)

탐지기	사용한 필드	이유
Port Scan	<code>IP.src/dst</code> , <code>TCP.dport</code> , <code>TCP.flags</code> , <code>UDP.sport/dport</code>	"누가 · 누구의 · 몇 개 포트"를 세고, 플래그로 연결 시작(SYN)과 스텔스 스캔을 구분

ARP Spoofing	ARP.op , psrc (Sender IP), hwsrc (Sender MAC), pdst , Ether.src	수신 호스트가 ARP Table에 반영하는 값이 Sender IP↔Sender MAC이므로 이것을 감시
DNS	DNSQR.qname , DNS.qr	질의 도메인 이름, 질의/응답 구분
ICMP	ICMP.type , Raw.load 길이	Echo Request(8) 빈도, payload 크기
HTTP / SSH	Raw.load , TCP.dport , TCP.flags	평문 요청 내용, SSH 배너(SSH-2.0-), 22번 포트 SYN 횟수

3. 탐지 기준

슬라이드 16의 두 방식을 섞었다. **Anomaly**(평소와 다른 양·변화)는 "시간 창 안의 개수가 임계값 이상"으로, **Signature**(알려진 패턴)는 문자열·플래그 비교로 구현했다. 모든 임계값은 config.py의 기본값을 config.json으로 바꿀 수 있다.

3-1. Port Scan — "짧은 시간에 다수 포트 접근"

규칙	기준	근거
PORT_SCAN_VERTICAL	10초 안에 한 출발지가 한 목적지의 서로 다른 포트 20개 이상 에 접근	슬라이드 18의 예시 규칙. 정상 클라이언트는 한 서버의 몇 개 포트만 사용한다
PORT_SCAN_HORIZONTAL	10초 안에 같은 포트로 서로 다른 호스트 15대 이상	22번 포트를 노리는 sweep은 수직 스캔 규칙으로는 안 잡힌다
STEALTH_SCAN_PACKET	TCP 플래그가 NULL / FIN 단독 / FIN+PSH+URG(XMAS)	정상 TCP에서는 나오지 않는 조합이라 패킷 하나로도 판단 가능 (Signature)

포트 접근은 **SYN이면서 ACK가 없는 패킷**(연결 시작)만 센다. UDP는 **출발지 포트가 1024 이상인 요청**만 센다. DNS 서버(53)가 클라이언트의 무작위 포트로 보내는 응답을 "다수 포트 접근"으로 착각하지 않도록 넣은 조건이다. 의심 쌍에 대해 SYN/ACK 응답을 따로 모아서, 공격자가 알아낸 **열린 포트**도 요약에 보여 준다.

3-2. ARP Spoofing — "IP-MAC 매핑 변화 감지"

IDS가 스스로 **신뢰 ARP 테이블**(trusted)과 **현재 보이는 매핑**(current)을 들고 있다가 새 ARP 패킷의 Sender IP/MAC과 비교한다. 게이트웨이는 config.json에 정적으로 등록할 수 있다(Static ARP와 같은 발상). 등록하지 않은 IP는 처음 본 매핑을 학습한다.

규칙	기준	근거
ARP_MAPPING_CHANGED	알고 있는 IP가 다른 MAC으로 나타남. 보호 IP(게이트웨이)면 HIGH. 그 MAC의 원래 주인 IP 를 공격자 후보로 표시	슬라이드 12의 탐지 아이디어 1. 공격자도 평소에 자기 IP로 ARP를 하므로 MAC을 역추적할 수 있다
ARP_REINFECTION	정상 MAC으로 recover된 뒤 다시 위조 MAC으로 바뀜	슬라이드 13 "ARP Recover — 시간 흐름을 봐야 한다"
ARP_REPLY_FLOOD	Request 없이 온 같은 Reply가 10초 안에 5회 이상	공격자는 감염을 유지하려고 infect 패킷을 반복한다(슬라이드 8). IDS를 켜기 전부터 감염돼 있어 "이전 상태"가 없을 때를 대비한 규칙
ARP_ETH_MISMATCH	Ethernet Src MAC ≠ ARP Sender MAC	정상 호스트는 두 값이 같다. 위조 도구의 흔적
ARP_DUPLICATE_MAC	이미 다른 IP의 주인인 MAC이 보호 IP를 처음 주장	학습 전에 게이트웨이를 사칭하는 경우

오탐 완화: 슬라이드 13에서 지적한 라우터 이중화는 `allowed_macs` 에 대기 라우터 MAC을 등록해 해결한다. ARP Probe(`psrc=0.0.0.0`)는 매핑 정보가 없으므로 무시한다.

3-3. 특정 패킷 — "DNS/ICMP/HTTP/SSH 조건 탐지"

대상	규칙	기준
DNS	DNS_BLOCKLIST	차단 목록 도메인과 그 하위 도메인 질의 (<code>a.evil.test</code> 는 잡고 <code>notevil.test</code> 는 통과)
	DNS_TUNNEL_SUSPECT	질의 길이 60자 초과, 또는 label 40자 초과이면서 새넨 엔트로피 ≥ 3.8 (무작위 문자열)
	DNS_SUBDOMAIN_BURST	10초 안에 같은 상위 도메인의 서로 다른 서브도메인 15개 이상 (터널링/DGA)
	DNS_QUERY_BURST	한 출발지가 10초 안에 DNS 질의 100회 이상 (LOW)
ICMP	ICMP_FLOOD	5초 안에 한 목적지로 Echo Request 50개 이상 (출발지가 여럿인 분산 flood 포함)
	ICMP_PING_SWEEP	10초 안에 서로 다른 호스트 10대 이상에 Echo Request
	ICMP_LARGE_PAYLOAD	payload 1000바이트 초과 (일반 ping은 32~56바이트) → ICMP 터널링 의심
HTTP	HTTP_ATTACK_PATTERN	URL 디코딩 후 <code>SQLi(union select)</code> / Path Traversal(<code>../</code>) / XSS(<code><script></code>) / 명령어 삽입 문자열
	HTTP_PLAINTEXT_CREDENTIAL	평문 요청에 <code>password=</code> , <code>Authorization: Basic</code> 등 포함 (2주차 HTTP vs HTTPS)
	HTTP_SCANNER_UA	User-Agent에 <code>sqlmap</code> , <code>nikto</code> , <code>nmap</code> 등 공격 도구 이름
SSH	SSH_BRUTE_FORCE	60초 안에 같은 서버 22번으로 새 연결(SYN) 10회 이상
	SSH_NONSTANDARD_PORT	22번이 아닌 포트에서 <code>SSH-</code> 배너 → 숨겨진 SSH 서버/터널 의심 (LOW)

같은 공격자에 대한 경고가 쏟아지지 않도록 규칙마다 **쿨다운**(기본 30초)을 둔다. 경고는 화면 출력과 함께 `-l alerts.jsonl` 로 JSON Lines 파일에 저장해 나중에 분석할 수 있다.

4. 핵심 코드

① 시간 창(State) — 모든 Anomaly 규칙의 기반 `mini_ids/detectors/base.py`

```
class SlidingWindow:
    """key 별로 (시각, 값)을 모아 두고 window 초보다 오래된 것은 버린다."""
    def add(self, key, ts, value=None):
        dq = self._data[key]
        dq.append((ts, value))
        while dq and ts - dq[0][0] > self.window: # 오래된 기록 제거
            dq.popleft()

    def distinct(self, key): # 창 안의 서로 다른 값 개수
        return len({v for _, v in self._data.get(key, ())})
```

② Port Scan — "누가 누구의 포트를 몇 개" mini_ids/detectors/portscan.py

```
is_probe, stealth = self._is_probe(p) # SYN(ACK 없음) / UDP 요청 / NULL·FIN·XMAS
if not is_probe or p.src_ip in self.whitelist:
    return
self.vertical.add((src, dst), ts, (p.proto, dport))
n_ports = self.vertical.distinct((src, dst))
if n_ports >= self.cfg["port_threshold"]: # 10초 안에 20개 이상
    self.suspects.add((src, dst))
    if self.cool.ready(("v", src, dst), ts):
        self.emit(Alert(ts, "PORTSCAN", "PORT_SCAN_VERTICAL", "MEDIUM", ...))
```

③ ARP — "현재 패킷 + 이전 상태" 비교 mini_ids/detectors/arp_spoof.py

```
known = self.trusted.get(ip) # IDS 가 믿고 있는 MAC
if known is None: # 처음 보는 IP → 학습
    self.trusted[ip] = mac; self.mac_owner[mac].add(ip); return
prev = self.current.get(ip, known)
if mac == known: # 정상 MAC
    if prev != known: self.recoveries[ip] += 1 # ARP recover 발생
    self.current[ip] = mac; return

# mac != known : 매핑이 바뀌었다 → 그 MAC 의 원래 주인 = 공격자 후보
attacker_of = sorted(self.mac_owner.get(mac, set()) - {ip})
if prev != mac: # 정상 → 위조로 넘어간 순간
    self.infections[ip] += 1
    if self.infections[ip] >= 2 and self.recoveries[ip] >= 1:
        self.emit(Alert(..., "ARP_REINFECTION", sev, ...)); return
self.emit(Alert(..., "ARP_MAPPING_CHANGED", sev,
    f"{ip}의 MAC 변경: {known} → {mac} (이 MAC의 원래 주인: {attacker_of})"))
```

④ HTTP — 2주차 Raw payload 검사 + URL 디코딩 mini_ids/detectors/packet_rules.py

```
if not p.payload.startswith((b"GET ", b"POST ", ...)): # 평문 HTTP 요청만
    return
text = unquote_plus(p.payload.decode(errors="ignore")).lower() # %27UNION%20SELECT → 'union select
for cat, pats in self.cfg["attack_patterns"].items():
    hit = next((pt for pt in pats if pt in text), None)
    if hit: self.emit(Alert(..., "HTTP_ATTACK_PATTERN", "HIGH", f"{cat} 패턴 '{hit}'"))
```

5. 실행 결과

테스트 트래픽은 2주차의 Packet Crafting(Ether()/IP()/TCP() + wrpcap)으로 만든 PCAP이다 (tools/gen_scenarios.py). 주소는 슬라이드와 같은 192.168.10.0/24 (게이트웨이 .1, 피해자 .2, 공격자 .3)를 쓰고, 외부 주소는 문서용 예약 대역을 썼다. 모든 공격 시나리오에는 정상 배경 트래픽(DNS, HTTPS, HTTP, ping, NTP)을 섞었다.

Port Scan — SYN 스캔 1024포트(S02), NULL/FIN/XMAS 스캔(S03)

```
$ python3 ids.py -r scenarios/S02_syn_scan.pcap -c config.json
[23:13:30] [ALERT] MEDIUM PORT_SCAN_VERTICAL 192.168.10.3 → 192.168.10.20: 10초 안에 서로 다른 TCP 포트 20개 접근

[*] 패킷 2189개 / 29.24초 분석, 경고 1건 {'PORT_SCAN_VERTICAL': 1}
$ python3 ids.py -r scenarios/S03_stealth_scan.pcap -c config.json
[23:13:28] [ALERT] MEDIUM STEALTH_SCAN_PACKET 192.168.10.3 → 192.168.10.20:632 비정상 TCP 플래그(NULL scan)
[23:13:28] [ALERT] MEDIUM PORT_SCAN_VERTICAL 192.168.10.3 → 192.168.10.20: 10초 안에 서로 다른 TCP 포트 20개 접근
[23:13:29] [ALERT] MEDIUM STEALTH_SCAN_PACKET 192.168.10.3 → 192.168.10.20:456 비정상 TCP 플래그(FIN scan)
[23:13:30] [ALERT] MEDIUM STEALTH_SCAN_PACKET 192.168.10.3 → 192.168.10.20:293 비정상 TCP 플래그(XMAS scan)

[*] 패킷 235개 / 29.62초 분석, 경고 4건 {'STEALTH_SCAN_PACKET': 3, 'PORT_SCAN_VERTICAL': 1}
```

요약(--summary)에는 공격자가 확인한 열린 포트도 나온다: 192.168.10.3 -> 192.168.10.20: probed_ports 1024, open_ports [22, 80, 443]

ARP Spoofing — 양방향 감염 → recover → 재감염 (S08)

```
$ python3 ids.py -r scenarios/S08_arp_spoofing.pcap -c config.json
[23:13:40] [ALERT] HIGH ARP_MAPPING_CHANGED 게이트웨이/보호 IP 192.168.10.1 의 MAC 변경: 02:00:00:aa:aa:aa → 02:00:00:cc:cc:cc (이 MAC 의 원래 주인: 192.168.10.3)
[23:13:40] [ALERT] MEDIUM ARP_MAPPING_CHANGED 192.168.10.2 의 MAC 변경: 02:00:00:bb:bb:bb → 02:00:00:c c:cc:cc (이 MAC 의 원래 주인: 192.168.10.3)
[23:13:48] [ALERT] HIGH ARP_REPLY_FLOOD 요청 없는 ARP Reply 반복: '192.168.10.1 is-at 02:00:00:c c:cc:cc' 10초 동안 5회 → 192.168.10.2
[23:13:48] [ALERT] MEDIUM ARP_REPLY_FLOOD 요청 없는 ARP Reply 반복: '192.168.10.2 is-at 02:00:00:c c:cc:cc' 10초 동안 5회 → 192.168.10.1
[23:14:15] [ALERT] HIGH ARP_REINFECTION 192.168.10.1 매핑이 recover 후 다시 02:00:00:cc:cc:cc 로 변경 (2번째 감염) (이 MAC 의 원래 주인: 192.168.10.3)
[23:14:15] [ALERT] MEDIUM ARP_REINFECTION 192.168.10.2 매핑이 recover 후 다시 02:00:00:cc:cc:cc 로 변경 (2번째 감염) (이 MAC 의 원래 주인: 192.168.10.3)
[23:14:23] [ALERT] HIGH ARP_REPLY_FLOOD 요청 없는 ARP Reply 반복: '192.168.10.1 is-at 02:00:00:c c:cc:cc' 10초 동안 5회 → 192.168.10.2
[23:14:23] [ALERT] MEDIUM ARP_REPLY_FLOOD 요청 없는 ARP Reply 반복: '192.168.10.2 is-at 02:00:00:c c:cc:cc' 10초 동안 5회 → 192.168.10.1

[*] 패킷 433개 / 79.34초 분석, 경고 8건 {'ARP_MAPPING_CHANGED': 2, 'ARP_REPLY_FLOOD': 4, 'ARP_REINFECTION': 2}
```

첫 경고에서 게이트웨이 IP가 공격자 MAC을 가리키게 된 순간을 HIGH로 잡았다. cc:cc:cc 의 원래 주인이 192.168.10.3 이라는 것까지 밝혀 슬라이드 9의 상황을 그대로 재현했다. 정상 ARP로 풀린 뒤 다시 감염되자 ARP_REINFECTION 이 울렸다.

특정 패킷 — DNS 터널링(S12), HTTP 공격(S17), 평문 로그인(S18), SSH Brute Force(S21)

```
$ python3 ids.py -r scenarios/S12_dns_tunnel.pcap -c config.json
[23:13:30] [ALERT] MEDIUM DNS_TUNNEL_SUSPECT 192.168.10.2 비정상적으로 긴/무작위 DNS 질의 (길이 70, 엔트로피 4.49): 52t2ikx753iteflzck3iw3olvjmayurahvkuvmdadf4fhuwi.0.t.tunnel-exfil.test...
[23:13:34] [ALERT] MEDIUM DNS_SUBDOMAIN_BURST 192.168.10.2 가 10초 동안 tunnel-exfil.test 의 서로 다른 서브도메인 15개 질의 (DNS 터널링/DGA 의심)

[*] 패킷 232개 / 39.64초 분석, 경고 2건 {'DNS_TUNNEL_SUSPECT': 1, 'DNS_SUBDOMAIN_BURST': 1}
$ python3 ids.py -r scenarios/S17_http_attacks.pcap -c config.json
[23:13:25] [ALERT] HIGH HTTP_ATTACK_PATTERN 192.168.10.3 → 192.168.10.20:80 SQL_INJECTION 패턴 'union select' : GET /item.php?id=1%27%20UNION%20SELECT%20user,password%20FROM%20users-- HTTP/1.1
[23:13:25] [ALERT] MEDIUM HTTP_SCANNER_UA 192.168.10.3 자동화 공격 도구 User-Agent: sqlmap/1.8#stable (https://sqlmap.org)
[23:13:28] [ALERT] HIGH HTTP_ATTACK_PATTERN 192.168.10.3 → 192.168.10.20:80 PATH_TRAVERSAL 패턴 '../' : GET /download?file=../../../../etc/passwd HTTP/1.1
[23:13:31] [ALERT] HIGH HTTP_ATTACK_PATTERN 192.168.10.3 → 192.168.10.20:80 XSS 패턴 '<script' : GET /search?q=%3Cscript%3Ealert(1)%3C/script%3E HTTP/1.1

[*] 패킷 194개 / 29.17초 분석, 경고 4건 {'HTTP_ATTACK_PATTERN': 3, 'HTTP_SCANNER_UA': 1}
$ python3 ids.py -r scenarios/S18_http_plain_login.pcap -c config.json
[23:13:30] [ALERT] MEDIUM HTTP_PLAINTEXT_CREDENTIAL 192.168.10.2 → 192.168.10.20:80 평문 HTTP 로 인증 정보 전송 ('password=' 포함)
[23:13:35] [ALERT] MEDIUM HTTP_PLAINTEXT_CREDENTIAL 192.168.10.2 → 192.168.10.20:80 평문 HTTP 로 인증 정보 전송 ('authorization: basic' 포함)

[*] 패킷 185개 / 29.88초 분석, 경고 2건 {'HTTP_PLAINTEXT_CREDENTIAL': 2}
$ python3 ids.py -r scenarios/S21_ssh_bruteforce.pcap -c config.json
[23:13:39] [ALERT] HIGH SSH_BRUTE_FORCE 192.168.10.3 → 192.168.10.20:22 60초 동안 SSH 연결 시도 10회

[*] 패킷 570개 / 59.12초 분석, 경고 1건 {'SSH_BRUTE_FORCE': 1}
```

실시간 캡처 테스트 (-i lo)

PCAP뿐 아니라 실제 캡처에서도 동작하는지 확인했다. IDS를 루프백에 붙여 두고 tools/live_demo_scan.py 로 127.0.0.1에 실제 TCP connect 스캔(1~300번 포트)과 평문 password 요청, SQLi 요청을 보냈다.

```
$ sudo python3 ids.py -i lo -t 20 --summary # 다른 터미널: python3 tools/live_demo_scan.py
[*] lo 에서 캡처 시작 (filter='arp or tcp or udp or icmp') - Ctrl+C 로 종료
[!] BPF 필터 사용 불가(Cannot set filter: libpcap is not available. Cannot compile filter !) → 필터 없이 캡처합니다
[19:34:35] [ALERT] MEDIUM PORT_SCAN_VERTICAL 127.0.0.1 → 127.0.0.1: 10초 안에 서로 다른 TCP 포트 20개 접근
[19:34:35] [ALERT] MEDIUM HTTP_PLAINTEXT_CREDENTIAL 127.0.0.1 → 127.0.0.1:8080 평문 HTTP 로 인증 정보 전송 ('password=' 포함)
[19:34:35] [ALERT] HIGH HTTP_ATTACK_PATTERN 127.0.0.1 → 127.0.0.1:8080 SQL_INJECTION 패턴 'union select' : GET /item?id=1%27%20UNION%20SELECT%20password%20FROM%20users-- HTTP/1.1

[*] 패킷 210개 / 0.03초 분석, 경고 3건 {'PORT_SCAN_VERTICAL': 1, 'HTTP_PLAINTEXT_CREDENTIAL': 1, 'HTTP_ATTACK_PATTERN': 1}
```

테스트 환경에 libpcap이 없어 BPF 필터를 컴파일하지 못했다. 이 경우 필터 없이 자동으로 다시 캡처하도록 수정했다(Windows에 Npcap이 없을 때도 같은 문제가 생긴다).

6. 테스트와 FP / FN 분석

시나리오마다 정답을 붙였다. 공격 시나리오는 **예상 규칙 중 하나라도 올리면 TP, 아니면 FN**이다. 정상 시나리오는 **경고가 하나라도 나오면 FP, 아니면 TN**이다. 두 설정으로 같은 PCAP을 돌렸다(tools/evaluate.py).

- **default:** 설정 파일 없음. 모든 매핑을 학습하고, 차단 도메인과 화이트리스트도 없음

- tuned:** config.json . 게이트웨이 MAC 정적 등록, 대기 라우터 MAC 허용, 차단 도메인, 모니터링 서버 화이트리스트, 백신 평판 조회 도메인 예외

15 TP (tuned)	3 FN (tuned)	1 FP (tuned)	6 TN (tuned)
-------------------------------------	------------------------------------	------------------------------------	------------------------------------

설정	TP	FN	FP	TN	탐지율 TP/(TP+FN)	오탐률 FP/(FP+TN)	정밀도 TP/(TP+FP)
default (설정 없음)	14	4	4	3	77.8%	57.1%	77.8%
tuned (config.json)	15	3	1	6	83.3%	14.3%	93.8%

#	시나리오	정답	default	tuned	올린 규칙 (tuned)
01	정상 사무실 트래픽 90초 (DNS 응답의 임의 목적지 포트, HTTPS 다중 연결 포함)	정상	TN	TN	-
02	nmap -sS 형태: 1~1024 포트 SYN 스캔 (약 2초)	공격	TP	TP	PORT_SCAN_VERTICAL
03	NULL / FIN / XMAS 스캔 각 30포트	공격	TP	TP	PORT_SCAN_VERTICAL, STEALTH_SCAN_PACKET
04	UDP 스캔 100포트	공격	TP	TP	PORT_SCAN_VERTICAL
05	수평 스캔: 192.168.10.100~139 의 22번 포트 sweep	공격	TP	TP	PORT_SCAN_HORIZONTAL
06	느린 스캔(nmap -T1 흉내): 3초에 1포트씩 60포트	공격	FN	FN	-
07	모니터링 서버가 5초 안에 서비스 포트 25개 헬스체크 (Nagios/Zabbix 형태)	정상	FP	TN	-
08	양방향 ARP Spoofing(infect 반복) + Relay + recover 후 재감염	공격	TP	TP	ARP_MAPPING_CHANGED, ARP_REINFECTION, ARP_REPLY_FLOOD
09	IDS 시작 전부터 감염 중 (첫 관측 매핑이 이미 위조)	공격	TP	TP	ARP_MAPPING_CHANGED, ARP_REINFECTION, ARP_REPLY_FLOOD
10	라우터 이중화(VRRP) 절체: 게이트웨이 IP 가 standby 라우터 MAC 으로 Gratuitous ARP	정상	FP	TN	-
11	DHCP 재할당: .50 을 쓰던 노트북이 떠나고 새 장비 가 같은 IP 사용	정상	FP	FP	ARP_MAPPING_CHANGED
12	DNS 터널링: 데이터를 base32 로 서브도메인에 실어 반복 질의	공격	TP	TP	DNS_SUBDOMAIN_BURST, DNS_TUNNEL_SUSPECT
13	악성코드 C2 도메인 질의 (malware-c2.example)	공격	FN	TP	DNS_BLOCKLIST
14	ICMP Flood: 2초 동안 Echo Request 200개	공격	TP	TP	ICMP_FLOOD
15	Ping Sweep: 192.168.10.1~60 에 Echo Request	공격	TP	TP	ICMP_PING_SWEEP
16	ICMP 터널링: payload 1400바이트 Echo 요청으로 데이터 전송	공격	TP	TP	ICMP_LARGE_PAYLOAD
17	SQLi(URL 인코딩)/Path Traversal/XSS + sqlmap User-Agent	공격	TP	TP	HTTP_ATTACK_PATTERN, HTTP_SCANNER_UA
18	평문 HTTP 로그인 (POST password=) + Basic 인증	공격	TP	TP	HTTP_PLAINTEXT_CREDENTIAL

19	같은 로그인을 HTTPS(TLS)로 전송	공격	FN	FN	-
20	SQLi 문자열을 두 TCP 세그먼트로 쪼개 전송	공격	FN	FN	-
21	SSH Brute Force: 1초 간격으로 40회 접속 시도 (hydra 형태)	공격	TP	TP	SSH_BRUTE_FORCE
22	관리자의 정상 SSH 접속 3회 (1분)	정상	TN	TN	-
23	비표준 포트(4444)에서 SSH 배너 → 백도어/터널 의심	공격	TP	TP	SSH_NONSTANDARD_PORT
24	뉴스 사이트 접속: 30초에 도메인 25개 + 같은 CDN 서버도메인 12개 질의	정상	TN	TN	-
25	백신의 DNS 기반 파일 평판 조회 (해시값이 서버도메인에 들어감)	정상	FP	TN	-

FN — 탐지하지 못한 공격과 원인

시나리오	원인
S06 느린 스캔 3초에 1포트, 60포트	10초 창 안에는 포트가 최대 4개만 들어와 임계값 20을 넘지 못한다. 슬라이드 17의 FN 예시("아주 천천히 포트 스캔해서 임계값 회피") 그대로다. 창을 넓히면 잡히지만 그만큼 정상 트래픽이 창 안에 쌓여 FP가 늘어난다.
S13 차단 도메인 (default)	차단 목록이 비어 있으면 Signature 규칙은 아무것도 모른다. config.json 에 도메인을 넣은 tuned 설정에서는 TP. 시그니처 방식은 목록의 품질이 곧 탐지 능력 이다.
S19 HTTPS 로그인	같은 로그인이라도 TLS로 암호화되면 password= 를 볼 수 없다. 2주차에서 배운 대로 IP·포트·크기 같은 메타데이터만 보인다. 사용자 입장에서는 이것이 오히려 올바른 보안 상태다.
S20 세그먼트 분할 SQLi	UNI / ON SEL / ECT 처럼 공격 문자열을 여러 TCP 세그먼트로 나누면 패킷 하나씩 검사하는 방식으로 패턴이 완성되지 않는다. 2주차 슬라이드의 "학습용 코드의 한계(TCP 재조립 없음)"를 실제로 확인한 사례다.

FP — 정상인데 경고한 경우와 원인

시나리오	원인과 조치
S07 모니터링 헬스체크 (default)	모니터링 서버가 5초 안에 서비스 포트 25개를 확인하는 동작은 모양만 보면 포트 스캔과 똑같다. whitelist_src 에 등록해 tuned에서 TN이 됐다. 단, 화이트리스트에 넣은 호스트가 탈취되면 그 호스트의 스캔은 보이지 않는다.
S10 라우터 이중화 (default)	VRRP 절체로 게이트웨이 IP의 MAC이 대기 라우터 MAC으로 바뀐 것을 공격으로 오인했다. 슬라이드 13/17의 FP 예시다. allowed_macs 등록으로 tuned에서 TN.
S11 DHCP 재할당 (두 설정 모두)	.50을 쓰던 노트북이 나가고 새 장비가 같은 IP를 받으면 정상적으로 IP-MAC이 바뀐다. 패킷만 봐서는 공격과 구별할 근거가 없다. 남은 유일한 FP 이고, DHCP 서버의 임대 기록과 연동해야 해결된다(개선점 참고).
S25 백신 평판 조회 (default)	일부 백신은 파일 해시를 서버도메인에 넣어 DNS로 조회한다. 길고 무작위인 label이라 터널링과 모양이 같다. allow_domains 예외로 tuned에서 TN.

튜닝 과정에서 발견한 것

- **DNS 질의 폭증 임계값:** 처음에는 10초에 40회로 정했다. 그런데 정상 뉴스 사이트 접속(S24, 도메인 37개 + 배경 트래픽)에서 FP가 나서 100회로 올렸다. 같은 데이터로 튜닝했으므로 새 환경에서는 다시 검증해야 한다.

- **ICMP Flood 집계 기준:** 처음에는 출발지별로 썼다. 그런데 Ping Sweep(S15)에서 Flood 경고까지 같이 올랐다. Flood는 "피해자가 받는 부하"이므로 **목적지 기준**으로 바꿨다. 덕분에 출발지가 여럿인 분산 flood도 잡힌다(단위 테스트로 확인).
- **감염 중에 IDS를 켜 경우(S09):** 학습형 IDS는 처음 본 맵핑을 믿는다. 그래서 위조 MAC을 "정상"으로 배웠다가, 진짜 게이트웨이 응답이 오자 **방향이 거꾸로 된 경고**를 냈다. 요청 없는 Reply가 반복되면(ARP_REPLY_FLOOD) 학습한 맵핑의 신뢰를 철회하도록 고쳤다. 그 뒤로는 올바른 방향(aa → cc)으로 재감염을 잡는다. 다만 첫 경고의 방향 오류는 남는다. 게이트웨이를 정적으로 등록하는 것(tuned)이 근본적인 해결책이다.
- **실시간 캡처 누락:** 루프백에서 300포트를 딜레이 없이 스캔하면(약 0.03초) IDS가 50포트(약 17%)만 봤다. 포트 사이에 2ms만 쉬어도 301포트를 모두 봤다. Python/Scapy의 패킷당 처리 속도가 한계라서, 고속 스캔(masscan 등)이나 트래픽이 많은 링크에서는 **캡처 단계에서 FN이 생길 수 있다**. 그래도 이번 테스트에서는 누락된 상태로도 임계값(20)을 넘어 탐지에는 성공했다.

7. 한계와 개선점

한계	개선 방향
느린 스캔 회피 (S06)	짧은 창(10초/20포트)과 긴 창(10분/50포트)을 함께 쓰는 다중 시간 창. 또는 출발지별로 "처음 접근한 포트" 누적 점수를 천천히 감소시키는 방식
TCP 재조립 없음 (S20)	(src, sport, dst, dport) 흐름별로 seq 순서대로 payload를 이어 붙인 뒤 검사. 또는 Scapy의 TCPSession 사용
암호화 트래픽 (S19)	payload 대신 메타데이터 활용: TLS SNI로 차단 도메인 검사, 비정상적으로 긴 세션·전송량, JA3 같은 TLS 지문
정상적인 IP-MAC 변경 (S11)	DHCP 패킷(ACK)을 함께 관찰해 "방금 임대된 IP의 새 MAC"은 정상 처리 (Switch의 DHCP Snooping + DAI와 같은 발상)
스위치 환경의 가시성	호스트 한 대에서 돌리면 브로드캐스트 ARP Request는 보이지만 다른 호스트로 가는 유니캐스트 Reply는 보이지 않는다. 미러링 포트(SPAN)에 IDS를 두거나, 각 호스트에서 자기 ARP 테이블(arp -a)을 주기적으로 비교하는 에이전트를 추가
Python 캡처 성능	BPF로 필요한 패킷만 받기, 파싱 최소화, 고속 환경에서는 Suricata/Zeek 같은 엔진에 규칙을 옮기기
고정 임계값	평소 트래픽을 일정 기간 학습해 호스트별 기준선(평균 + 표준편차)을 만드는 적응형 임계값
탐지만 가능 (IDS)	ARP 위조를 탐지하면 정상 Reply를 보내 피해자를 recover시키거나, 스캐너 IP를 방화벽에 등록하는 대응(IPS 방향)

정리 — 탐지 규칙을 만드는 일보다 **"언제 틀리는가"**를 확인하는 일이 더 많은 것을 알게 됐다. 기본 설정의 오탐률 57%가 환경 정보(게이트웨이 MAC, 이중화 라우터, 모니터링 서버)를 넣자 14%로 줄었다. 결국 슬라이드의 마지막 문장처럼, 탐지는 **"정상 상태를 알고 달라진 지점을 찾는 것"**이고, 정상 상태를 얼마나 정확히 알려 주느냐가 IDS의 품질을 결정했다.

부록. 재현 방법

```

pip install scapy
python3 tools/gen_scenarios.py                # 테스트 PCAP 25개 + 정답(manifest.json) 생성
python3 tools/evaluate.py                    # TP/FP/FN/TN 집계 → results/
python3 -m unittest discover -s tests -v     # 단위 테스트 21개
python3 ids.py -r scenarios/S08_arp_spoofing.pcap -c config.json --summary
sudo python3 ids.py -i eth0 -c config.json -l alerts.jsonl # 실시간

```

모든 테스트 트래픽은 PCAP 안의 가상 패킷이거나 127.0.0.1로만 보낸 것이다. 스캔·스푸핑은 본인 소유이거나 허가받은 실습망에서만 수행해야 한다.